

Chapter 16: Query Optimization

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use



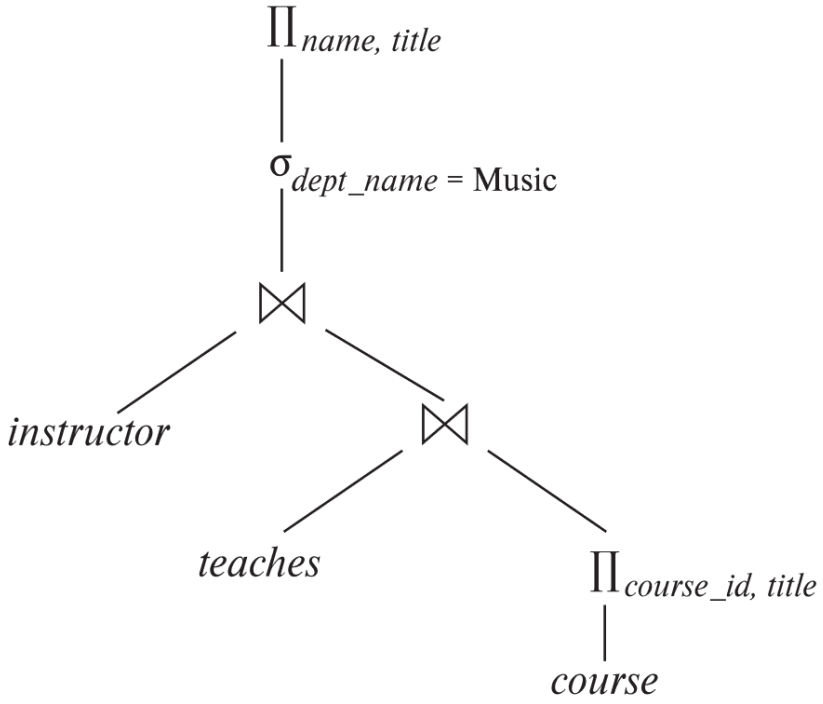
Chapter 16: Query Optimization

- Introduction
- Transformation of Relational Expressions
- Catalog Information for Cost Estimation
- Statistical Information for Cost Estimation
- Cost-based optimization
- Dynamic Programming for Choosing Evaluation Plans
- Materialized views

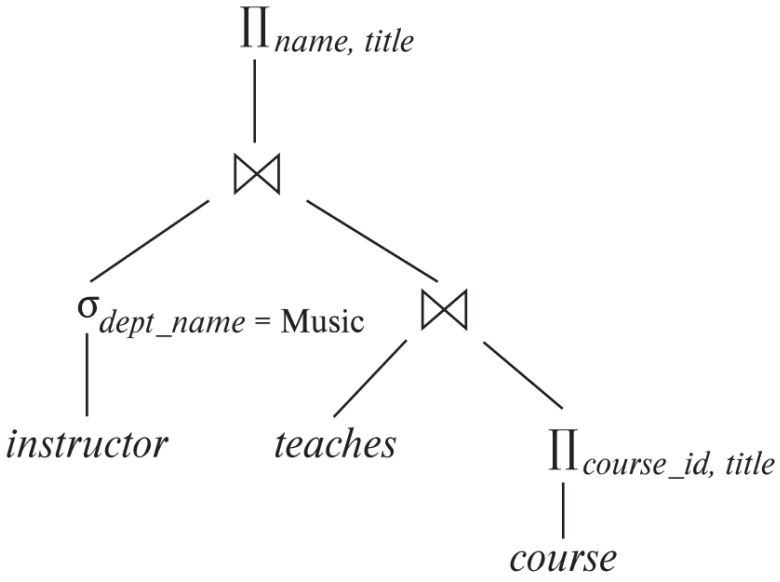


Introduction

- Alternative ways of evaluating a given query
  - Equivalent expressions
  - Different algorithms for each operation



(a) Initial expression tree



(b) Transformed expression tree

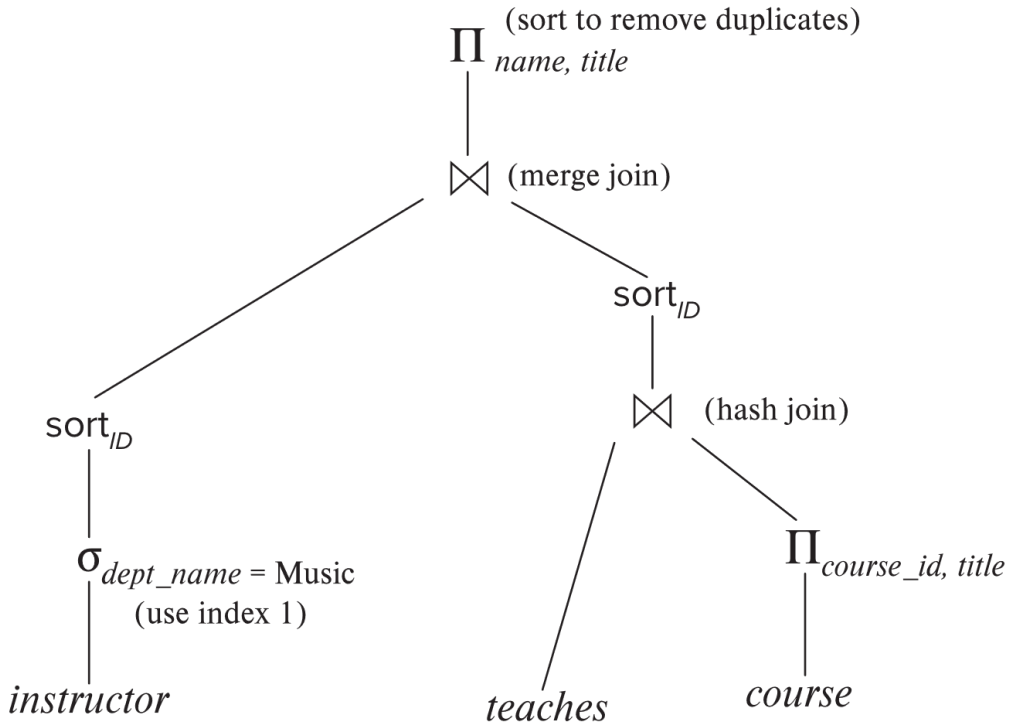
©Silberschatz, Korth and Sudarshan



Introduction (Cont.)



- An **evaluation plan** defines exactly what algorithm is used for each operation, and how the execution of the operations is coordinated.



- Find out how to view query execution plans on your favorite database



Introduction (Cont.)



- Cost difference between evaluation plans for a query can be enormous
  - E.g. seconds vs. days in some cases
- Steps in **cost-based query optimization**
  1. Generate logically equivalent expressions using **equivalence rules**
  2. Annotate resultant expressions to get alternative query plans
  3. Choose the cheapest plan based on **estimated cost**
- Estimation of plan cost based on:
  - Statistical information about relations. Examples:
    - number of tuples, number of distinct values for an attribute
  - Statistics estimation for intermediate results
    - to compute cost of complex expressions
  - Cost formulae for algorithms, computed using statistics

©Silberschatz, Korth and Sudarshan



Viewing Query Evaluation Plans



- Most database support **explain** <query>
  - Displays plan chosen by query optimizer, along with cost estimates
  - Some syntax variations between databases
    - Oracle: **explain plan for** <query> followed by **select** * **from** table (*dbms_xplan.display*)
    - SQL Server: **set showplan_text on**
- Some databases (e.g. PostgreSQL) support **explain analyse** <query>
  - Shows actual runtime statistics found by running the query, in addition to showing the plan
- Some databases (e.g. PostgreSQL) show cost as *f..l*
  - *f* is the cost of delivering first tuple and *l* is cost of delivering all results

©Silberschatz, Korth and Sudarshan



Generating Equivalent Expressions

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use



Transformation of Relational Expressions



- Two relational algebra expressions are said to be **equivalent** if the two expressions generate the same set of tuples on every *legal* database instance
  - Note: order of tuples is irrelevant
  - we don't care if they generate different results on databases that violate integrity constraints
- In SQL, inputs and outputs are multisets of tuples
  - Two expressions in the multiset version of the relational algebra are said to be equivalent if the two expressions generate the same multiset of tuples on every legal database instance.
- An **equivalence rule** says that expressions of two forms are equivalent
  - Can replace expression of first form by second, or vice versa

©Silberschatz, Korth and Sudarshan



Equivalence Rules

1. Conjunctive selection operations can be deconstructed into a sequence of individual selections.

$$\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

2. Selection operations are commutative.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

3. Only the last in a sequence of projection operations is needed, the others can be omitted.

$$\Pi_{L_1}(\Pi_{L_2}(\ldots(\Pi_{L_n}(E))\ldots)) \equiv \Pi_{L_1}(E)$$

where $L_1 \subseteq L_2 \ldots \subseteq L_n$

4. Selections can be combined with Cartesian products and theta joins.

   a. $\sigma_{\theta}(E_1 \times E_2) \equiv E_1 \bowtie_{\theta} E_2$

   b. $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) \equiv E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$

©Silberschatz, Korth and Sudarshan



Equivalence Rules (Cont.)

5. Theta-join operations (and natural joins) are commutative.

$$E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$$

6. (a) Natural join operations are associative:

$$(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$$

(b) Theta joins are associative in the following manner:

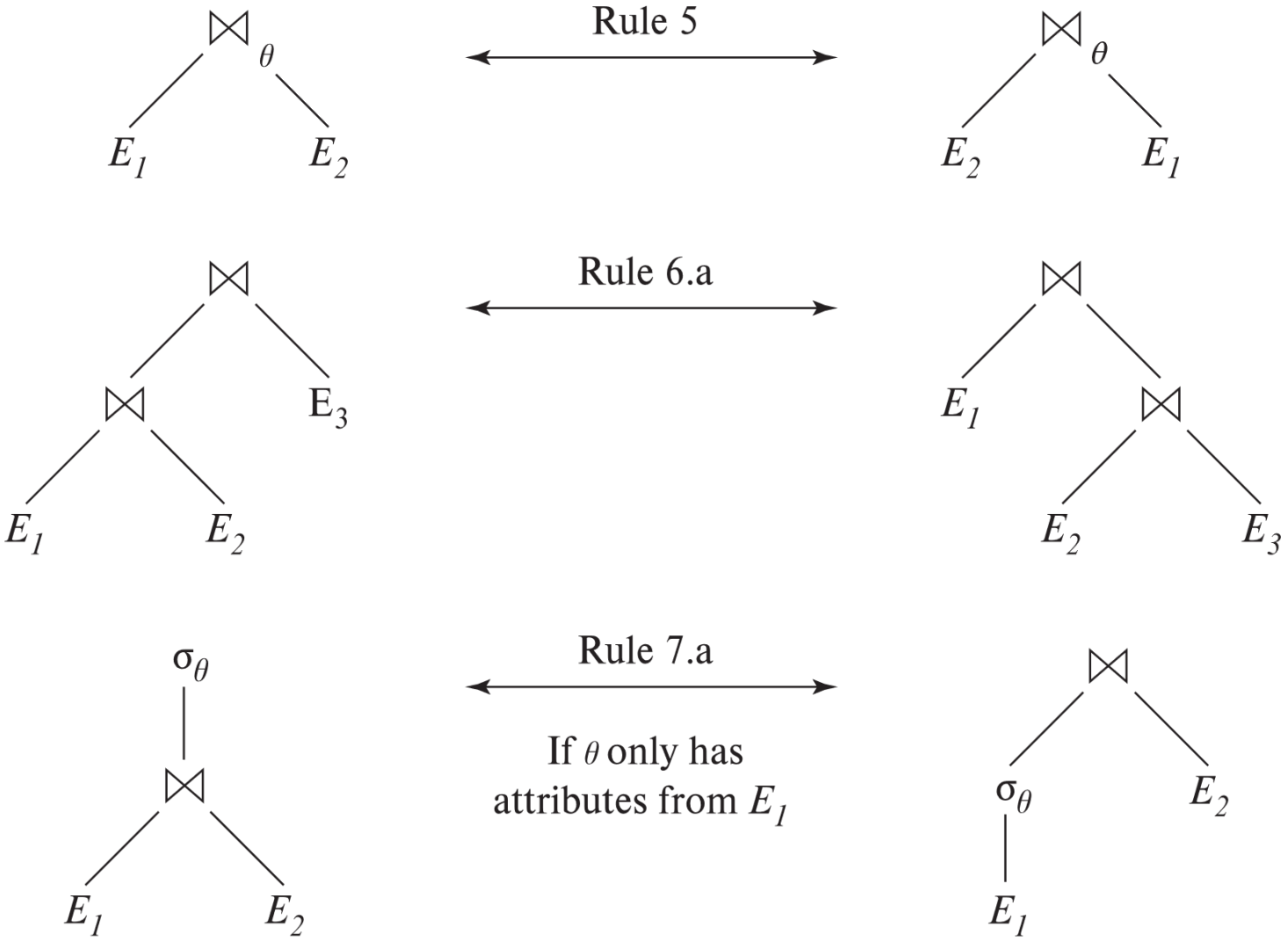
$$(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 \equiv E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$$

where θ_2 involves attributes from only E_2 and E_3.

©Silberschatz, Korth and Sudarshan



Pictorial Depiction of Equivalence Rules



 ©Silberschatz, Korth and Sudarshan



Equivalence Rules (Cont.)

7. The selection operation distributes over the theta join operation under the following two conditions:
   (a) When all the attributes in θ_0 involve only the attributes of one of the expressions (E_1) being joined.

$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) \quad \equiv \quad (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

   (b) When θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2.

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) \quad \equiv \quad (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$$

©Silberschatz, Korth and Sudarshan



Equivalence Rules (Cont.)

8. The projection operation distributes over the theta join operation as follows:

   (a) if θ involves only attributes from $L_1 \cup L_2$:

   $$\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) \equiv \Pi_{L_1}(E_1) \bowtie_\theta \Pi_{L_2}(E_2)$$

   (b) In general, consider a join $E_1 \bowtie_\theta E_2$.

   - Let L_1 and L_2 be sets of attributes from E_1 and E_2, respectively.
   - Let L_3 be attributes of E_1 that are involved in join condition θ, but are not in $L_1 \cup L_2$, and
   - let L_4 be attributes of E_2 that are involved in join condition θ, but are not in $L_1 \cup L_2$.

   $$\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) \equiv \Pi_{L_1 \cup L_2}(\Pi_{L_1 \cup L_3}(E_1) \bowtie_\theta \Pi_{L_2 \cup L_4}(E_2))$$

Similar equivalences hold for outerjoin operations: $\mathbin{⟕}$, $\mathbin{⟖}$, and $\mathbin{⟗}$

©Silberschatz, Korth and Sudarshan



Equivalence Rules (Cont.)

9. The set operations union and intersection are commutative

$$E_1 \cup E_2 \equiv E_2 \cup E_1$$
$$E_1 \cap E_2 \equiv E_2 \cap E_1$$

(set difference is not commutative).

10. Set union and intersection are associative.

$$(E_1 \cup E_2) \cup E_3 \equiv E_1 \cup (E_2 \cup E_3)$$
$$(E_1 \cap E_2) \cap E_3 \equiv E_1 \cap (E_2 \cap E_3)$$

11. The selection operation distributes over $\cup$, $\cap$ and $-$.

a. $\sigma_\theta (E_1 \cup E_2) \equiv \sigma_\theta (E_1) \cup \sigma_\theta(E_2)$
b. $\sigma_\theta (E_1 \cap E_2) \equiv \sigma_\theta (E_1) \cap \sigma_\theta(E_2)$
c. $\sigma_\theta (E_1 - E_2) \equiv \sigma_\theta (E_1) - \sigma_\theta(E_2)$
d. $\sigma_\theta (E_1 \cap E_2) \equiv \sigma_\theta(E_1) \cap E_2$
e. $\sigma_\theta (E_1 - E_2) \equiv \sigma_\theta(E_1) - E_2$

preceding equivalence does not hold for $\cup$

12. The projection operation distributes over union

$$\Pi_L(E_1 \cup E_2) \equiv (\Pi_L(E_1)) \cup (\Pi_L(E_2))$$

©Silberschatz, Korth and Sudarshan



Exercise

- Create equivalence rules involving
  - The group by/aggregation operation
  - Left outer join operation



Equivalence Rules (Cont.)

13. Selection distributes over aggregation as below

$$\sigma_\theta({}_G\gamma_A(E)) \equiv {}_G\gamma_A(\sigma_\theta(E))$$

provided θ only involves attributes in G

14. a. Full outerjoin is commutative:

$$E_1 \mathbin{⟗} E_2 \equiv E_2 \mathbin{⟗} E_1$$

b. Left and right outerjoin are not commutative, but:

$$E_1 ⟕ E_2 \equiv E_2 ⟖ E_1$$

15. Selection distributes over left and right outerjoins as below, provided θ_1 only involves attributes of E_1

a. $\sigma_{\theta_1}(E_1 ⟕_\theta E_2) \equiv (\sigma_{\theta_1}(E_1)) ⟕_\theta E_2$

b. $\sigma_{\theta_1}(E_1 ⟖_\theta E_2) \equiv E_2 ⟕_\theta (\sigma_{\theta_1}(E_1))$

16. Outerjoins can be replaced by inner joins under some conditions

a. $\sigma_{\theta_1}(E_1 ⟕_\theta E_2) \equiv \sigma_{\theta_1}(E_1 ⋈_\theta E_2)$

b. $\sigma_{\theta_1}(E_1 ⟖_\theta E_1) \equiv \sigma_{\theta_1}(E_1 ⋈_\theta E_2)$

provided θ_1 is null rejecting on E_2

©Silberschatz, Korth and Sudarshan



Equivalence Rules (Cont.)

Note that several equivalences that hold for joins do not hold for outerjoins

- $\sigma_{year=2017}(instructor \bowtie teaches) \not\equiv \sigma_{year=2017}(instructor \bowtie teaches)$
- Outerjoins are not associative

$$(r \bowtie s) \bowtie t \quad \not\equiv \quad r \bowtie (s \bowtie t)$$

  - e.g. with r(A,B) = {(1,1), s(B,C) = { (1,1)}, t(A,C) = { }

©Silberschatz, Korth and Sudarshan



Transformation Example: Pushing Selections

- Query: Find the names of all instructors in the Music department, along with the titles of the courses that they teach
  - $\Pi_{name,\ title}(\sigma_{dept_name = \text{'Music'}}(instructor \bowtie (teaches \bowtie \Pi_{course_id,\ title}(course))))$
- Transformation using rule 7a.
  - $\Pi_{name,\ title}((\sigma_{dept_name = \text{'Music'}}(instructor)) \bowtie (teaches \bowtie \Pi_{course_id,\ title}(course)))$
- Performing the selection as early as possible reduces the size of the relation to be joined.

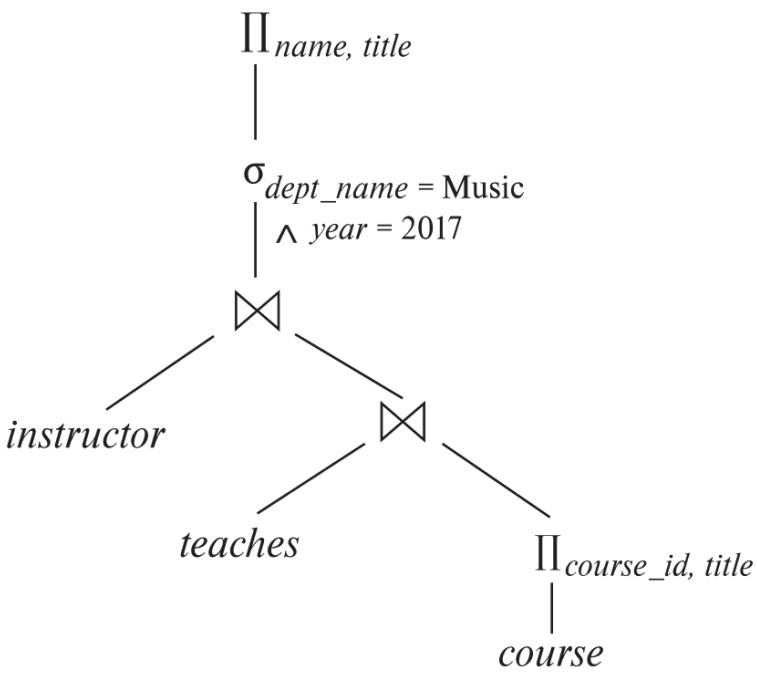
©Silberschatz, Korth and Sudarshan



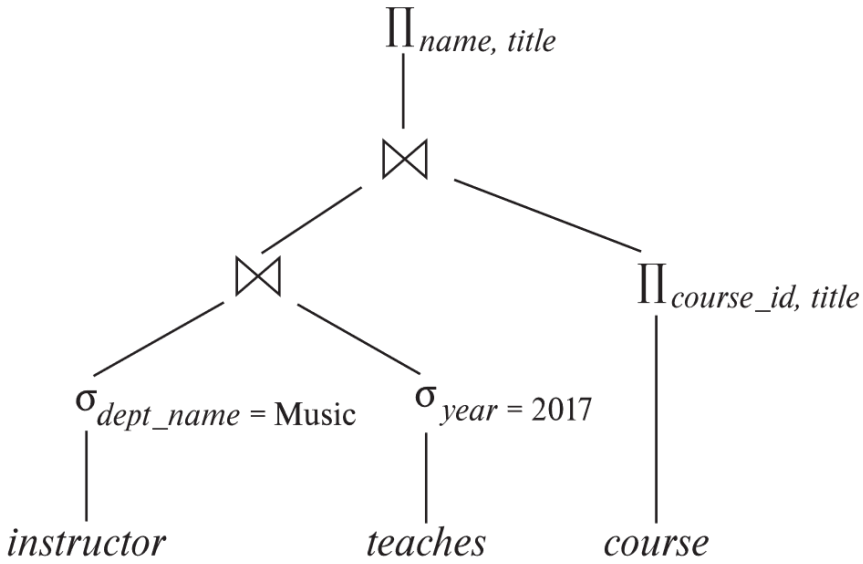
Multiple Transformations (Cont.)



تحديد الكوست الأقل



(a) Initial expression tree



(b) Tree after multiple transformations



Join Ordering Example

- For all relations r_1, r_2, and r_3,

$$(r_1 \bowtie r_2) \bowtie r_3 = r_1 \bowtie (r_2 \bowtie r_3)$$

  (Join Associativity) $\bowtie$

- If $r_2 \bowtie r_3$ is quite large and $r_1 \bowtie r_2$ is small, we choose

$$(r_1 \bowtie r_2) \bowtie r_3$$

  so that we compute and store a smaller temporary relation.

©Silberschatz, Korth and Sudarshan



Join Ordering Example (Cont.)

- Consider the expression

$$\Pi_{name,\ title}(\sigma_{dept_name=\text{“Music”}}(instructor) \bowtie teaches) \bowtie \Pi_{course_id,\ title}(course))))$$

- Could compute $teaches \bowtie \Pi_{course_id,\ title}(course)$ first, and join result with

$$\sigma_{dept_name=\text{“Music”}}(instructor)$$

but the result of the first join is likely to be a large relation.

- Only a small fraction of the university's instructors are likely to be from the Music department
  - it is better to compute

$$\sigma_{dept_name=\text{“Music”}}(instructor) \bowtie teaches$$

first.

©Silberschatz, Korth and Sudarshan



Enumeration of Equivalent Expressions

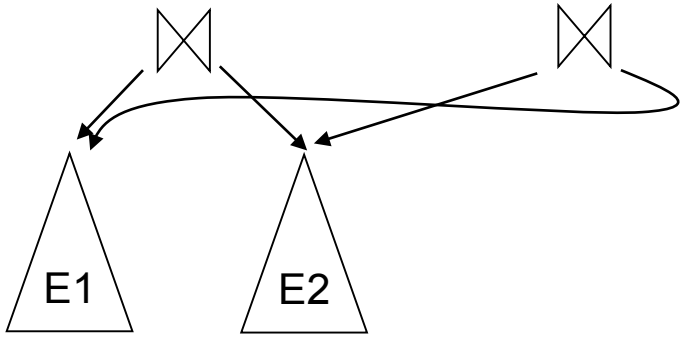
- Query optimizers use equivalence rules to **systematically** generate expressions equivalent to the given expression
- Can generate all equivalent expressions as follows:
  - Repeat
    - apply all applicable equivalence rules on every subexpression of every equivalent expression found so far
    - add newly generated expressions to the set of equivalent expressions

    Until no new equivalent expressions are generated above
- The above approach is very expensive in space and time
  - Two approaches
    - Optimized plan generation based on transformation rules
    - Special case approach for queries with only selections, projections and joins



Implementing Transformation Based Optimization

- Space requirements reduced by sharing common sub-expressions:
  - when E1 is generated from E2 by an equivalence rule, usually only the top level of the two are different, subtrees below are the same and can be shared using pointers
    - E.g. when applying join commutativity



  - Same sub-expression may get generated multiple times
    - Detect duplicate sub-expressions and share one copy
- Time requirements are reduced by not generating all expressions
  - Dynamic programming
    - We will study only the special case of dynamic programming for join order optimization

 ©Silberschatz, Korth and Sudarshan



Cost Estimation

- Cost of each operator computer as described in Chapter 15
  - Need statistics of input relations
    - E.g. number of tuples, sizes of tuples
- Inputs can be results of sub-expressions
  - Need to estimate statistics of expression results
  - To do so, we require additional statistics
    - E.g. number of distinct values for an attribute
- More on cost estimation later

©Silberschatz, Korth and Sudarshan



Choice of Evaluation Plans

- Must consider the interaction of evaluation techniques when choosing evaluation plans
  - choosing the cheapest algorithm for each operation independently may not yield best overall algorithm. E.g.
    - merge-join may be costlier than hash-join, but may provide a sorted output which reduces the cost for an outer level aggregation.
    - nested-loop join may provide opportunity for pipelining
- Practical query optimizers incorporate elements of the following two broad approaches:
  1. Search all the plans and choose the best plan in a cost-based fashion.
  2. Uses heuristics to choose a plan.

©Silberschatz, Korth and Sudarshan



Cost-Based Optimization

- Consider finding the best join-order for $r_1 \bowtie r_2 \bowtie \ldots \bowtie r_n$.
- There are $(2(n-1))!/(n-1)!$ different join orders for above expression. With $n = 7$, the number is 665280, with $n = 10$, the number is greater than 176 billion!
- No need to generate all the join orders. Using dynamic programming, the least-cost join order for any subset of $\{r_1, r_2, \ldots r_n\}$ is computed only once and stored for future use.

©Silberschatz, Korth and Sudarshan



Dynamic Programming in Optimization

- To find best join tree for a set of *n* relations:
  - To find best plan for a set *S* of *n* relations, consider all possible plans of the form: $S_1 \bowtie (S - S_1)$ where S_1 is any non-empty subset of *S*.
  - Recursively compute costs for joining subsets of *S* to find the cost of each plan. Choose the cheapest of the $2^n - 2$ alternatives.
  - Base case for recursion: single relation access plan
    - Apply all selections on R_i using best choice of indices on R_i
  - When plan for any subset is computed, store it and reuse it when it is required again, instead of recomputing it
    - Dynamic programming

©Silberschatz, Korth and Sudarshan



Join Order Optimization Algorithm

```
procedure findbestplan(S)
    if (bestplan[S].cost ≠ ∞)
            return bestplan[S]
    // else bestplan[S] has not been computed earlier, compute it now
    if (S contains only 1 relation)
        set bestplan[S].plan and bestplan[S].cost based on the best way
        of accessing S  using selections on S and indices (if any) on S
    else for each non-empty subset S1 of S such that S1 ≠ S
            P1= findbestplan(S1)
            P2= findbestplan(S - S1)
            for each algorithm A for joining results of P1 and P2
                … compute plan and cost of using A (see next page) ..
                if cost < bestplan[S].cost
                        bestplan[S].cost = cost
                        bestplan[S].plan = plan;
    return bestplan[S]
```



Join Order Optimization Algorithm (cont.)

for each algorithm A for joining results of *P*1 and *P*2
// For indexed-nested loops join, the outer could be *P1* or *P2*
// Similarly for hash-join, the build relation could be *P1* or *P2*
// We assume the alternatives are considered as separate algorithms

if algorithm *A* is indexed nested loops
Let P_i and P_o denote inner and outer inputs
if P_i has a single relation r_i and r_i has an index on the join attribute

plan = "execute P_o.*plan*; join results of P_o and r_i using *A*",
with any selection conditions on P_i performed as part of the join condition

cost = P_o.*cost* + cost of *A*

else *cost* = ∞; /* cannot use indexed nested loops join */

else
plan = "execute *P1.plan*; execute *P2.plan*;
join results of *P1* and *P2* using *A*;"
cost = *P1.cost* + *P2.cost* + cost of *A*

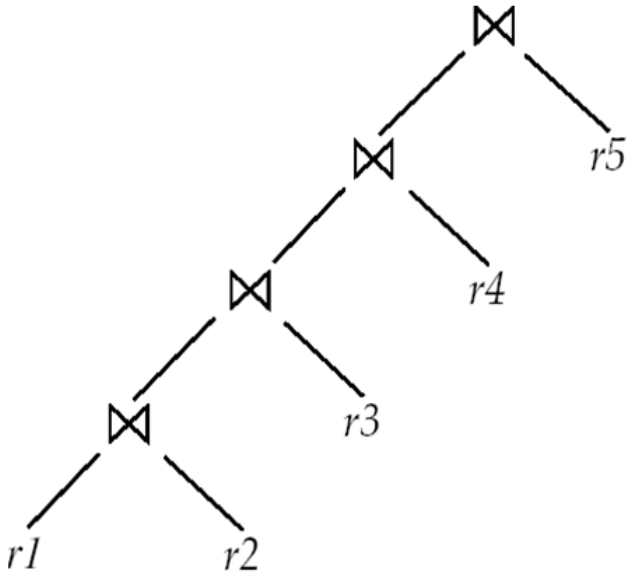
…. *See previous page*

 ©Silberschatz, Korth and Sudarshan

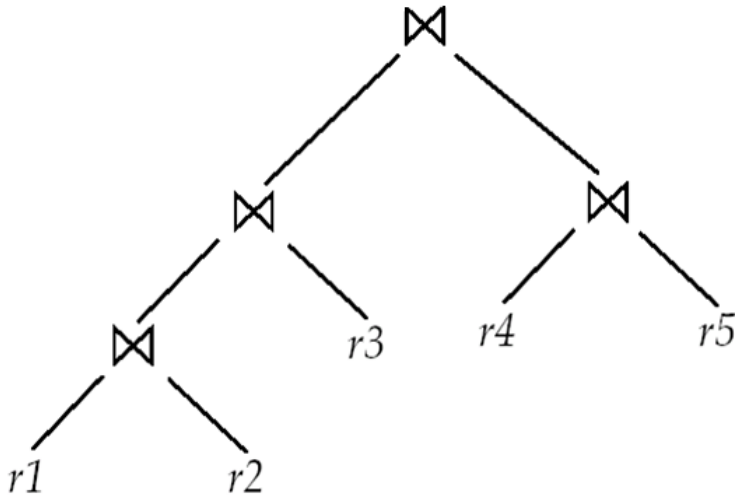


Left Deep Join Trees

- In **left-deep join trees,** the right-hand-side input for each join is a relation, not the result of an intermediate join.



(a) Left-deep join tree



(b) Non-left-deep join tree

 ©Silberschatz, Korth and Sudarshan



Cost of Optimization

- With dynamic programming time complexity of optimization with bushy trees is $O(3^n)$.
  - With n = 10, this number is 59000 instead of 176 billion!
- Space complexity is $O(2^n)$
- To find best left-deep join tree for a set of n relations:
  - Consider n alternatives with one relation as right-hand side input and the other relations as left-hand side input.
  - Modify optimization algorithm:
    - Replace "**for each** non-empty subset $S1$ of S such that $S1 \neq S$"
    - By: **for each** relation r in S
let S1 = S – r .
- If only left-deep trees are considered, time complexity of finding best join order is $O(n\ 2^n)$
  - Space complexity remains at $O(2^n)$
- Cost-based optimization is expensive, but worthwhile for queries on large datasets (typical queries have small n, generally < 10)

©Silberschatz, Korth and Sudarshan



Interesting Sort Orders

- Consider the expression $(r_1 \bowtie r_2) \bowtie r_3$ (with A as common attribute)
- An **interesting sort order** is a particular sort order of tuples that could make a later operation (join/group by/order by) cheaper
  - Using merge-join to compute $r_1 \bowtie r_2$ may be costlier than hash join but generates result sorted on A
  - Which in turn may make merge-join with r_3 cheaper, which may reduce cost of join with r_3 and minimizing overall cost
- Not sufficient to find the best join order for each subset of the set of n given relations
  - must find the best join order for each subset, **for each interesting sort order**
  - Simple extension of earlier dynamic programming algorithms
  - Usually, number of interesting orders is quite small and doesn't affect time/space complexity significantly

©Silberschatz, Korth and Sudarshan



Cost Based Optimization with Equivalence Rules

- **Physical equivalence rules** allow logical query plan to be converted to physical query plan specifying what algorithms are used for each operation.
- Efficient optimizer based on equivalent rules depends on
  - A space efficient representation of expressions which avoids making multiple copies of subexpressions
  - Efficient techniques for detecting duplicate derivations of expressions
  - A form of dynamic programming based on **memoization**, which stores the best plan for a subexpression the first time it is optimized, and reuses in on repeated optimization calls on same subexpression
  - Cost-based pruning techniques that avoid generating all plans
- Pioneered by the Volcano project and implemented in the SQL Server optimizer

©Silberschatz, Korth and Sudarshan



Heuristic Optimization

- Cost-based optimization is expensive, even with dynamic programming.
- Systems may use *heuristics* to reduce the number of choices that must be made in a cost-based fashion.
- Heuristic optimization transforms the query-tree by using a set of rules that typically (but not in all cases) improve execution performance:
  - Perform selection early (reduces the number of tuples)
  - Perform projection early (reduces the number of attributes)
  - Perform most restrictive selection and join operations (i.e. with smallest result size) before other similar operations.
  - Some systems use only heuristics, others combine heuristics with partial cost-based optimization.

©Silberschatz, Korth and Sudarshan



Structure of Query Optimizers

- Many optimizers considers only left-deep join orders.
  - Plus heuristics to push selections and projections down the query tree
  - Reduces optimization complexity and generates plans amenable to pipelined evaluation.
- Heuristic optimization used in some versions of Oracle:
  - Repeatedly pick “best” relation to join next
    - Starting from each of n starting points. Pick best among these
- Intricacies of SQL complicate query optimization
  - E.g. nested subqueries



Structure of Query Optimizers (Cont.)

- Some query optimizers integrate heuristic selection and the generation of alternative access plans.
  - Frequently used approach
    - heuristic rewriting of nested block structure and aggregation
    - followed by cost-based join-order optimization for each block
  - Some optimizers (e.g. SQL Server) apply transformations to entire query and do not depend on block structure
  - **Optimization cost budget** to stop optimization early (if cost of plan is less than cost of optimization)
  - **Plan caching** to reuse previously computed plan if query is resubmitted
    - Even with different constants in query
- Even with the use of heuristics, cost-based query optimization imposes a substantial overhead.
  - But is worth it for expensive queries
  - Optimizers often use simple heuristics for very cheap queries, and perform exhaustive enumeration for more expensive queries

©Silberschatz, Korth and Sudarshan



Statistics for Cost Estimation

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use



Statistical Information for Cost Estimation

- n_r: number of tuples in a relation *r.*
- b_r: number of blocks containing tuples of *r.*
- l_r: size of a tuple of *r.*
- f_r: blocking factor of *r* — i.e., the number of tuples of *r* that fit into one block.
- *V(A, r):* number of distinct values that appear in *r* for attribute *A;* same as the size of $\prod_A(r)$.
- If tuples of *r* are stored together physically in a file, then:

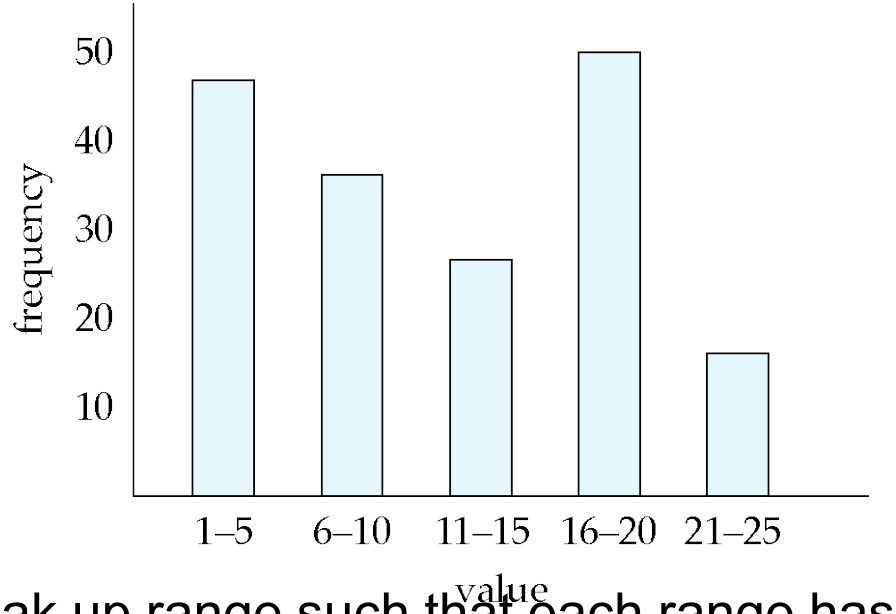
$$b_r = \left\lceil \frac{n_r}{f_r} \right\rceil$$

©Silberschatz, Korth and Sudarshan



Histograms

- Histogram on attribute *age* of relation *person*



- **Equi-width** histograms
- **Equi-depth** histograms break up range such that each range has (approximately) the same number of tuples
  - E.g. (4, 8, 14, 19)
- Many databases also store *n* **most-frequent values** and their counts
  - Histogram is built on remaining values only

©Silberschatz, Korth and Sudarshan



Histograms (cont.)

- Histograms and other statistics usually computed based on a **random sample**
- Statistics may be out of date
  - Some database require a **analyze** command to be executed to update statistics
  - Others automatically recompute statistics
    - e.g. when number of tuples in a relation changes by some percentage



Selection Size Estimation

- $\sigma_{A=v}(r)$
  - $n_r / V(A,r)$: number of records that will satisfy the selection
  - Equality condition on a key attribute: *size estimate* = 1
- $\sigma_{A \leq V}(r)$ (case of $\sigma_{A \geq V}(r)$ is symmetric)
  - Let c denote the estimated number of tuples satisfying the condition.
  - If min(A,r) and max(A,r) are available in catalog
    - c = 0 if v < min(A,r)
    - c = $n_r \cdot \frac{v - \min(A,r)}{\max(A,r) - \min(A,r)}$
  - If histograms available, can refine above estimate
  - In absence of statistical information *c* is assumed to be $n_r / 2$.

©Silberschatz, Korth and Sudarshan



Size Estimation of Complex Selections

- The **selectivity** of a condition θ_i is the probability that a tuple in the relation *r* satisfies θ_i .
  - If s_i is the number of satisfying tuples in *r,* the selectivity of θ_i is given by s_i / n_r.
- **Conjunction:** $\sigma_{\theta 1 \wedge \theta 2 \wedge \ldots \wedge \theta n}(r)$. *Assuming independence,* estimate of tuples in the result is: $n_r * \frac{s_1 * s_2 * \ldots * s_n}{n_r^n}$
- **Disjunction:** $\sigma_{\theta 1 \vee \theta 2 \vee \ldots \vee \theta n}(r)$. Estimated number of tuples:

$$n_r * \left(1 - (1 - \frac{s_1}{n_r}) * (1 - \frac{s_2}{n_r}) * \ldots * (1 - \frac{s_n}{n_r})\right)$$

- **Negation:** $\sigma_{\neg\theta}(r)$. Estimated number of tuples:

$n_r - size(\sigma_\theta(r))$



Join Operation: Running Example

Running example:

$student \bowtie takes$

Catalog information for join examples:

- $n_{student}$ = 5,000.
- $f_{student}$ = 50, which implies that $b_{student}$ =5000/50 = 100.
- n_{takes} = 10000.
- f_{takes} = 25, which implies that b_{takes} = 10000/25 = 400.
- *V(ID, takes)* = 2500, which implies that on average, each student who has taken a course has taken 4 courses.
  - Attribute *ID* in *takes* is a foreign key referencing *student.*
  - *V*(*ID, student*) = 5000 (*primary key!*)



Estimation of the Size of Joins

- The Cartesian product r x s contains $n_r . n_s$ tuples; each tuple occupies $s_r + s_s$ bytes.
- If $R \cap S = \varnothing$, then $r \bowtie s$ is the same as r x s.
- If $R \cap S$ is a key for R, then a tuple of s will join with at most one tuple from r
  - therefore, the number of tuples in $r \bowtie s$ is no greater than the number of tuples in s.
- If $R \cap S$ *in* S is a foreign key in S referencing R, then the number of tuples in $r \bowtie s$ is exactly the same as the number of tuples in s.
    - The case for $R \cap S$ being a foreign key referencing S is symmetric.
- In the example query *student* $\bowtie$ *takes, ID* in *takes* is a foreign key referencing *student*
  - hence, the result has exactly n_{takes} tuples, which is 10000

©Silberschatz, Korth and Sudarshan



Estimation of the Size of Joins (Cont.)

- If $R \cap S = \{A\}$ is not a key for R or S.
  If we assume that every tuple t in R produces tuples in $R \bowtie S$, the number of tuples in $R \bowtie S$ is estimated to be:

$$\frac{n_r * n_s}{V(A, s)}$$

  If the reverse is true, the estimate obtained will be:

$$\frac{n_r * n_s}{V(A, r)}$$

  The lower of these two estimates is probably the more accurate one.
- Can improve on above if histograms are available
  - Use formula similar to above, for each cell of histograms on the two relations



Estimation of the Size of Joins (Cont.)

- Compute the size estimates for *depositor* ⋈ *customer* without using information about foreign keys:
  - *V(ID, takes)* = 2500, and
    V(ID, student) = 5000
  - The two estimates are 5000 * 10000/2500 = 20,000 and 5000 * 10000/5000 = 10000
  - We choose the lower estimate, which in this case, is the same as our earlier computation using foreign keys.

©Silberschatz, Korth and Sudarshan



Size Estimation for Other Operations

- Projection: estimated size of $\Pi_A(r)$ = $V(A,r)$
- Aggregation : estimated size of ${}_G\gamma_A(r)$ = $V(G,r)$
- Set operations
  - For unions/intersections of selections on the same relation: rewrite and use size estimate for selections
    - E.g. $\sigma_{\theta 1}(r) \cup \sigma_{\theta 2}(r)$ can be rewritten as $\sigma_{\theta 1 \text{ or } \theta 2}(r)$
  - For operations on different relations:
    - estimated size of $r \cup s$ = size of r + size of s.
    - estimated size of $r \cap s$ = minimum size of r and size of s.
    - estimated size of $r - s$ = r.
    - All the three estimates may be quite inaccurate, but provide upper bounds on the sizes.

©Silberschatz, Korth and Sudarshan



Size Estimation (Cont.)

- Outer join:
  - Estimated size of $r \mathbin{⟕} s$ = *size of* $r \bowtie s$ + *size of* r
    - Case of right outer join is symmetric
  - Estimated size of $r \mathbin{⟗} s$ = *size of* $r \bowtie s$ + size of r + size of s

©Silberschatz, Korth and Sudarshan



Estimation of Number of Distinct Values

Selections: $\sigma_\theta(r)$

- If θ forces A to take a specified value: $V(A,\sigma_\theta(r)) = 1$.
  - e.g., $A = 3$
- If θ forces A to take on one of a specified set of values:
  $V(A,\sigma_\theta(r))$ = number of specified values.
  - (e.g., $(A = 1 \ V \ A = 3 \ V \ A = 4)$),
- If the selection condition θ is of the form $A \ op \ r$
  estimated $V(A,\sigma_\theta(r)) = V(A.r) * s$
  - where s is the selectivity of the selection.
- In all the other cases: use approximate estimate of
  $\min(V(A,r), n_{\sigma\theta\,(r)})$
  - More accurate estimate can be got using probability theory, but this one works fine generally

©Silberschatz, Korth and Sudarshan



Estimation of Distinct Values (Cont.)

Joins: $r \bowtie s$

- If all attributes in A are from r
  estimated $V(A, r \bowtie s) = \min(V(A,r), n_{r \bowtie s})$
- If A contains attributes $A1$ from r and $A2$ from s, then estimated $V(A, r \bowtie s)$ =

$$\min(V(A1,r)*V(A2 - A1,s), V(A1 - A2,r)*V(A2,s), n_{r \bowtie s})$$

  - More accurate estimate can be got using probability theory, but this one works fine generally

©Silberschatz, Korth and Sudarshan



Estimation of Distinct Values (Cont.)

- Estimation of distinct values are straightforward for projections.
  - They are the same in $\Pi_{A\,(r)}$ as in *r*.
- The same holds for grouping attributes of aggregation.
- For aggregated values
  - For min(*A*) and max(*A*), the number of distinct values can be estimated as min(V(*A,r*), *V*(*G,r*)) where G denotes grouping attributes
  - For other aggregates, assume all values are distinct, and use *V*(*G,r*)



ADDITIONAL OPTIMIZATION TECHNIQUES

- Nested Subqueries
- Materialized Views

©Silberschatz, Korth and Sudarshan



Optimizing Nested Subqueries**

- Nested query example:

  select *name*
  from *instructor*
  where exists (**select** *
  from *teaches*
  where *instructor.ID = teaches.ID* **and** *teaches.year = 2019*)

- SQL conceptually treats nested subqueries in the where clause as functions that take parameters and return a single value or set of values
  - Parameters are variables from outer level query that are used in the nested subquery; such variables are called **correlation variables**
- Conceptually, nested subquery is executed once for each tuple in the cross-product generated by the outer level **from** clause
  - Such evaluation is called **correlated evaluation**
  - Note: other conditions in where clause may be used to compute a join (instead of a cross-product) before executing the nested subquery



Optimizing Nested Subqueries (Cont.)

- Correlated evaluation may be quite inefficient since
  - a large number of calls may be made to the nested query
  - there may be unnecessary random I/O as a result
- SQL optimizers attempt to transform nested subqueries to joins where possible, enabling use of efficient join techniques
- E.g.: earlier nested query can be rewritten as

  $\Pi_{name}(instructor \bowtie_{instructor.ID=teaches.ID \wedge teaches.year=2019} teaches)$

- Note: the two queries generate different numbers of duplicates (why?)
  - Can be modified to handle duplicates correctly using semijoins

©Silberschatz, Korth and Sudarshan



Optimizing Nested Subqueries (Cont.)

- The **semijoin** operator $\ltimes$ is defined as follows
  - A tuple r_i appears n times in $r \ltimes_\theta s$ if it appears n times in r, and there is at least one matching tuple s_i in s
- E.g.: earlier nested query can be rewritten as

  $\Pi_{name}(instructor \ltimes_{instructor.ID=teaches.ID \wedge teaches.year=2019} teaches)$
  - Or even as: $\Pi_{name}(instructor \ltimes_{instructor.ID=teaches.ID} (\sigma_{teaches.year=2019} teaches))$
  - Now the duplicate count is correct!
- The above relational algebra query is also equivalent to

  from *instructor*
  where *ID* **in** (**select** *teaches.ID*
  from *teaches*
  where *teaches.year = 2019*)



Optimizing Nested Subqueries (Cont.)

- This could also be written using only joins (in SQL) as

  with t_1 **as**
  (**select distinct** *ID*
  from *teaches*
  where *year = 2019)*
  select *name*
  from *instructor*, t_1
  where t_1.*ID* = *instructor.ID*

- The query

  select *name*
  from *instructor*
  where not exists (**select** *
  from *teaches*
  where *instructor.ID* = *teaches.ID* **and** *teaches.year* = *2019*)

  can be rewritten using the **anti-semijoin** operation as $\overline{\ltimes}$

$$\Pi_{name}(instructor \ \overline{\ltimes}_{instructor.ID=teaches.ID \wedge teaches.year=2019} \ teaches)$$



Optimizing Nested Subqueries (Cont.)

In general, SQL queries of the form below can be rewritten as shown

- Rewrite: **select** A
  from $r_1, r_2, \ldots, r_n$
  where P_1 **and exists** (**select** *
  from $s_1, s_2, \ldots, s_m$
  where P_2^1 **and** P_2^2)
- To: $\Pi_A(\sigma_{P1}(r_1 \times r_2 \times \ldots \times r_n) \ltimes_{P2^2} \sigma_{P2^1}(s_1 \times s_2 \times \ldots \times s_m)$
  - P_2^1 contains predicates that do not involve any correlation variables
  - P_2^2 contains predicates involving correlation variables
- The process of replacing a nested query by a query with a join/semijoin (possibly with a temporary relation) is called **decorrelation**.
- Decorrelation is more complicated in several cases, e.g.
  - The nested subquery uses aggregation, or
  - The nested subquery is a scalar subquery
  - Correlated evaluation used in these cases



Decorrelation (Cont.)

- Decorrelation of scalar aggregate subqueries can be done using groupby/aggregation in some cases
- **select** *name*
  from *instructor*
  where 1 < (**select count**(*)
  from *teaches*
  where *instructor.ID = teaches.ID*
  and *teaches.year = 2019*)
- $\Pi_{name}(instructor \ltimes_{instructor.ID=TID \wedge 1 < cnt} ($
  $ID\ as\ TID\ \gamma_{\mathbf{count}(*)\ as\ cnt}(\sigma_{teaches.year=2019}(teaches))))$



Materialized Views

- A **materialized view** is a view whose contents are computed and stored.
- Consider the view
  c**reate view** *department_total_salary*(*dept_name, total_salary*) **as**
  select *dept_name*, **sum**(*salary*)
  from *instructor*
  group by *dept_name*
- Materializing the above view would be very useful if the total salary by department is required frequently
  - Saves the effort of finding multiple tuples and adding up their amounts

©Silberschatz, Korth and Sudarshan



Materialized View Maintenance

- The task of keeping a materialized view up-to-date with the underlying data is known as **materialized view maintenance**
- Materialized views can be maintained by recomputation on every update
- A better option is to use **incremental view maintenance**
  - **Changes to database relations are used to compute changes to the materialized view, which is then updated**
- View maintenance can be done by
  - Manually defining triggers on insert, delete, and update of each relation in the view definition
  - Manually written code to update the view whenever database relations are updated
  - Periodic recomputation (e.g. nightly)
  - Incremental maintenance supported by many database systems
    - Avoids manual effort/correctness issues



Incremental View Maintenance

- The changes (inserts and deletes) to a relation or expressions are referred to as its **differential**
  - Set of tuples inserted to and deleted from r are denoted $\mathbf{i}_r$ and $\mathbf{d}_r$
- To simplify our description, we only consider inserts and deletes
  - We replace updates to a tuple by deletion of the tuple followed by insertion of the update tuple
- We describe how to compute the change to the result of each relational operation, given changes to its inputs
- We then outline how to handle relational algebra expressions

©Silberschatz, Korth and Sudarshan



Join Operation

- Consider the materialized view $v = r \bowtie s$ and an update to r
- Let r^{old} and r^{new} denote the old and new states of relation r
- Consider the case of an insert to r:
  - We can write $r^{new} \bowtie s$ as $(r^{old} \cup i_r) \bowtie s$
  - And rewrite the above to $(r^{old} \bowtie s) \cup (i_r \bowtie s)$
  - But $(r^{old} \bowtie s)$ is simply the old value of the materialized view, so the incremental change to the view is just $i_r \bowtie s$
- Thus, for inserts $v^{new} = v^{old} \cup (i_r \bowtie s)$
- Similarly for deletes $v^{new} = v^{old} - (d_r \bowtie s)$

A, 1 B, 2	⋈	1, p 2, r 2, s
C,2		

A, 1, p B, 2, r B, 2, s
C, 2, r C, 2, s



Selection and Projection Operations

- Selection: Consider a view $v = \sigma_\theta(r)$.
  - $v^{new} = v^{old} \cup \sigma_\theta(i_r)$
  - $v^{new} = v^{old} - \sigma_\theta(d_r)$
- Projection is a more difficult operation
  - $R = (A,B)$, and r(R) = { (a,2), (a,3)}
  - $\prod_A(r)$ has a single tuple (a).
  - If we delete the tuple (a,2) from r, we should not delete the tuple (a) from $\prod_A(r)$, but if we then delete (a,3) as well, we should delete the tuple
- For each tuple in a projection $\prod_A(r)$, we will keep a count of how many times it was derived
  - On insert of a tuple to r, if the resultant tuple is already in $\prod_A(r)$ we increment its count, else we add a new tuple with count = 1
  - On delete of a tuple from r, we decrement the count of the corresponding tuple in $\prod_A(r)$
    - if the count becomes 0, we delete the tuple from $\prod_A(r)$

©Silberschatz, Korth and Sudarshan



Aggregation Operations

- **Count** : $v = {}_A\gamma_{count(B)}^{(r)}$.
  - When a set of tuples i_r is inserted
    - For each tuple r in i_r, if the corresponding group is already present in v, we increment its count, else we add a new tuple with count = 1
  - When a set of tuples d_r is deleted
    - for each tuple t in i_r we look for the group *t.A* in *v*, and subtract 1 from the count for the group.
      - If the count becomes 0, we delete from *v* the tuple for the group *t.A*
- **Sum**: $v = {}_A\gamma_{sum\ (B)}^{(r)}$
  - We maintain the sum in a manner similar to count, except we add/subtract the B value instead of adding/subtracting 1 for the count
  - Additionally we maintain the count in order to detect groups with no tuples. Such groups are deleted from v
    - Cannot simply test for sum = 0 (why?)
- **Avg**: How to handle average?
  - Maintain **sum** and **count** separately, and divide at the end

©Silberschatz, Korth and Sudarshan



Aggregate Operations (Cont.)

- **min**, **max**: $v = {}_{A}\gamma_{min\,(B)}(r)$.
  - Handling insertions on r is straightforward.
  - Maintaining the aggregate values **min** and **max** on deletions may be more expensive. We have to look at the other tuples of *r* that are in the same group to find the new minimum



Other Operations

- Set intersection: $v = r \cap s$
  - when a tuple is inserted in *r* we check if it is present in *s*, and if so we add it to *v*.
  - If the tuple is deleted from r, we delete it from the intersection if it is present.
  - Updates to *s* are symmetric
  - The other set operations, *union* and *set difference* are handled in a similar fashion.
- Outer joins are handled in much the same way as joins but with some extra work
  - we leave details to you.

©Silberschatz, Korth and Sudarshan



Handling Expressions

- To handle an entire expression, we derive expressions for computing the incremental change to the result of each sub-expressions, starting from the smallest sub-expressions.
- E.g. consider $E_1 \bowtie E_2$ where each of E_1 and E_2 may be a complex expression
  - Suppose the set of tuples to be inserted into E_1 is given by D_1
    - Computed earlier, since smaller sub-expressions are handled first
  - Then the set of tuples to be inserted into $E_1 \bowtie E_2$ is given by $D_1 \bowtie E_2$
    - This is just the usual way of maintaining joins

©Silberschatz, Korth and Sudarshan



Query Optimization and Materialized Views

- Rewriting queries to use materialized views:
  - A materialized view $v = r \bowtie s$ is available
  - A user submits a query $r \bowtie s \bowtie t$
  - We can rewrite the query as $v \bowtie t$
    - Whether to do so depends on cost estimates for the two alternative
- Replacing a use of a materialized view by the view definition:
  - A materialized view $v = r \bowtie s$ is available, but without any index on it
  - User submits a query $\sigma_{A=10}(v)$.
  - Suppose also that *s* has an index on the common attribute B, and r has an index on attribute A.
  - The best plan for this query may be to replace *v* by $r \bowtie s$, which can lead to the query plan $\sigma_{A=10}(r) \bowtie s$
- Query optimizer should be extended to consider all above alternatives and choose the best overall plan



Materialized View Selection

- **Materialized view selection**: "What is the best set of views to materialize?"
- **Index selection**: "what is the best set of indices to create"
  - closely related, to materialized view selection
    - but simpler
- Materialized view selection and index selection based on typical system **workload** (queries and updates)
  - Typical goal: minimize time to execute workload , subject to constraints on space and time taken for some critical queries/updates
  - One of the steps in database tuning
    - more on tuning in later chapters
- Commercial database systems provide tools (called "tuning assistants" or "wizards") to help the database administrator choose what indices and materialized views to create

©Silberschatz, Korth and Sudarshan



Additional Optimization Techniques

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use



Top-K Queries

- **Top-K queries**

  select *
  from r, s
  where r.B = s.B
  order by r.A **ascending**
  limit 10

  - Alternative 1: Indexed nested loops join with r as outer
  - Alternative 2: estimate highest r.A value in result and add selection (**and** r.A <= H) to where clause
    - If < 10 results, retry with larger H

©Silberschatz, Korth and Sudarshan



Optimization of Updates

- **Halloween problem**

  update R **set** A = 5 * A
  where A > 10

  - If index on A is used to find tuples satisfying A > 10, and tuples updated immediately, same tuple may be found (and updated) multiple times
  - Solution 1: *Always defer updates*
    - collect the updates (old and new values of tuples) and update relation and indices in second pass
    - Drawback: extra overhead even if e.g. update is only on R.B, not on attributes in selection condition
  - Solution 2: *Defer only if required*
    - Perform immediate update if update does not affect attributes in where clause, and deferred updates otherwise.

©Silberschatz, Korth and Sudarshan



Join Minimization

- **Join minimization**

  select r.A, r.B
  from r, s
  where r.B = s.B

- Check if join with s is redundant, drop it
  - E.g. join condition is on foreign key from r to s, r.B is declared as not null, and no selection on s
  - Other sufficient conditions possible

    select r.A, s2.B
    from r, s **as** s1, s **as** s2
    where r.B=s1.B **and** r.B = s2.B **and** s1.A < 20 **and** s2.A < 10

    - join with s1 is redundant and can be dropped (along with selection on s1)
  - Lots of research in this area since 70s/80s!

©Silberschatz, Korth and Sudarshan



Multiquery Optimization

- Example

  Q1: **select** * **from** (r **natural join** t) **natural join** s

  Q2: **select** * **from** (r **natural join** u) **natural join** s

  - Both queries share common subexpression (r natural join s)
  - May be useful to compute (r natural join s) once and use it in both queries
    - But this may be more expensive in some situations
      - e.g. (r natural join s) may be expensive, plans as shown in queries may be cheaper
- **Multiquery optimization**: find best overall plan for a set of queries, expoiting sharing of common subexpressions between queries where it is useful

©Silberschatz, Korth and Sudarshan



Multiquery Optimization (Cont.)

- Simple heuristic used in some database systems:
  - optimize each query separately
  - detect and exploiting common subexpressions in the individual optimal query plans
    - May not always give best plan, but is cheap to implement
  - **Shared scans**: widely used special case of multiquery optimization
- Set of materialized views may share common subexpressions
  - As a result, view maintenance plans may share subexpressions
  - Multiquery optimization can be useful in such situations

©Silberschatz, Korth and Sudarshan



Parametric Query Optimization

- Example
  select *
  from r **natural join** s
  where r.a < $1
  - value of parameter $1 not known at compile time
    - known only at run time
  - different plans may be optimal for different values of $1
- Solution 1: optimize at run time, each time query is submitted
    - can be expensive
- Solution 2: **Parametric Query Optimization**:
  - optimizer generates a set of plans, optimal for different values of $1
    - Set of optimal plans usually small for 1 to 3 parameters
    - Key issue: how to do find set of optimal plans efficiently
  - best one from this set is chosen at run time when $1 is known
- Solution 3: **Query Plan Caching**
  - If optimizer decides that same plan is likely to be optimal for all parameter values, it caches plan and reuses it, else reoptimize each time
  - Implemented in many database systems



Plan Stability Across Optimizer Changes

- What if 95% of plans are faster on database/optimizer version N+1 than on N, but 5% are slower?
  - Why should plans be slower on new improved optimizer?
    - Answer: Two wrongs can make a right, fixing one wrong can make things worse!
- Approaches:
  - Allow hints for tuning queries
    - Not practical for migrating large systems with no access to source code
  - Set optimization level, default to version N (Oracle)
    - And migrate one query at a time after testing both plans on new optimizer
  - Save plan from version N, and give it to optimizer version N+1
    - Sybase, XML representation of plans (SQL Server)

©Silberschatz, Korth and Sudarshan



Adaptive Query Processing

- Some systems support adaptive operators that change execution algorithm on the fly
  - E.g. (indexed) nested loops join or hash join chosen at run time, depending on size of outer input
- Other systems allow monitoring of behavior of plan at run time and adapt plan
  - E.g. if statistics such as number of rows is found to be very different in reality from what optimizer estimated
  - Can stop execution, compute fresh plan, and restart
    - But must avoid too many such restarts

©Silberschatz, Korth and Sudarshan

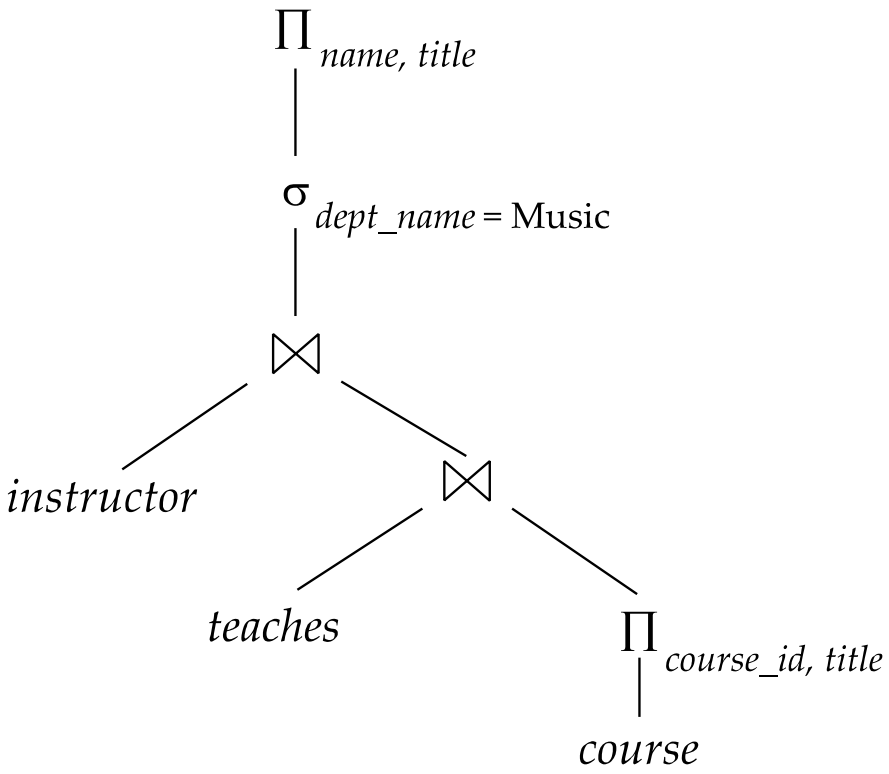


End of Chapter

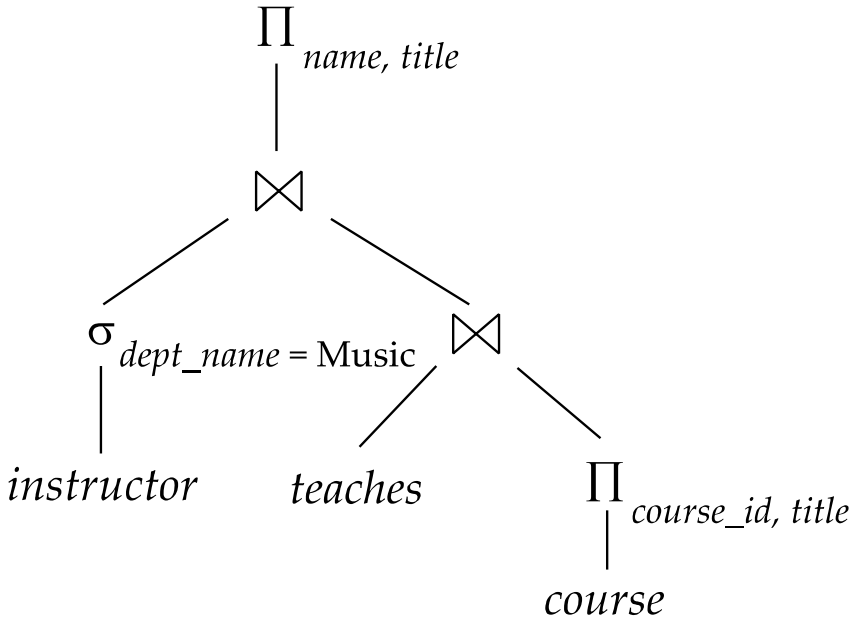
©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use



Figure 13.01



(a) Initial expression tree



(b) Transformed expression tree



Figure 13.02

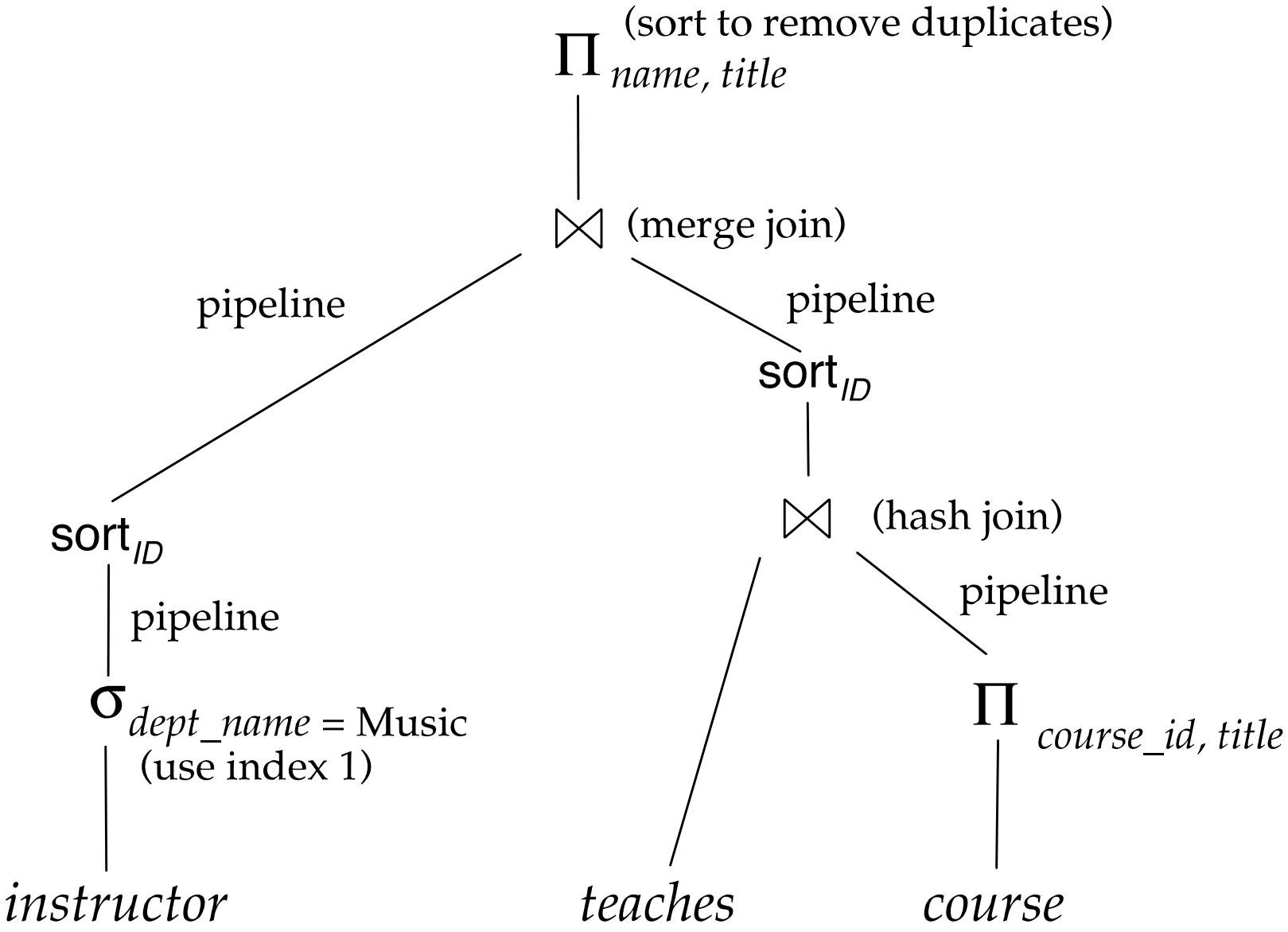

Π (sort to remove duplicates)
name, title
⋈ (merge join)
pipeline
pipeline
sort ID
sort ID
⋈ (hash join)
pipeline
pipeline
σ dept_name = Music
(use index 1)
Π course_id, title
instructor
teaches
course



Figure 13.03

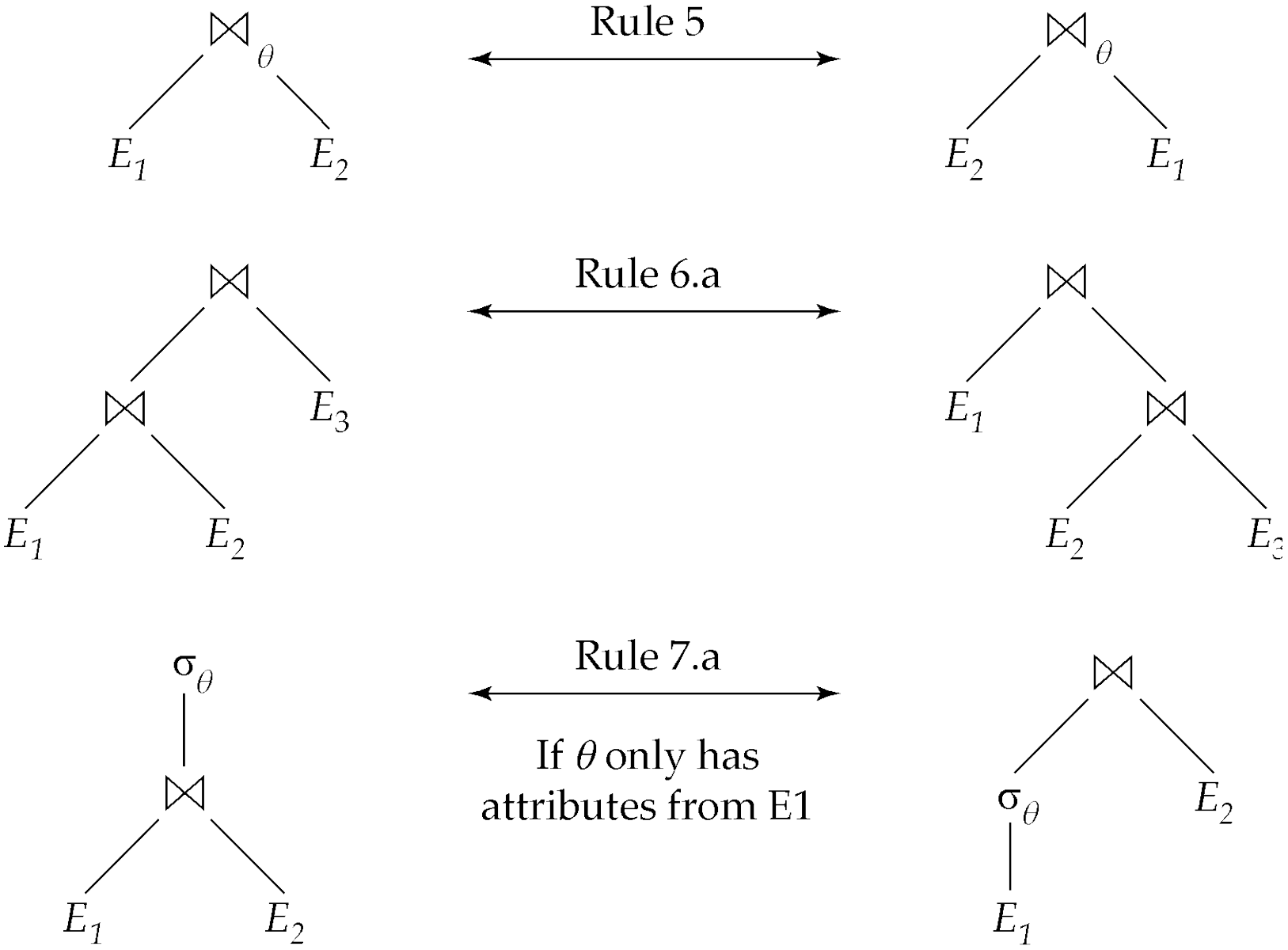
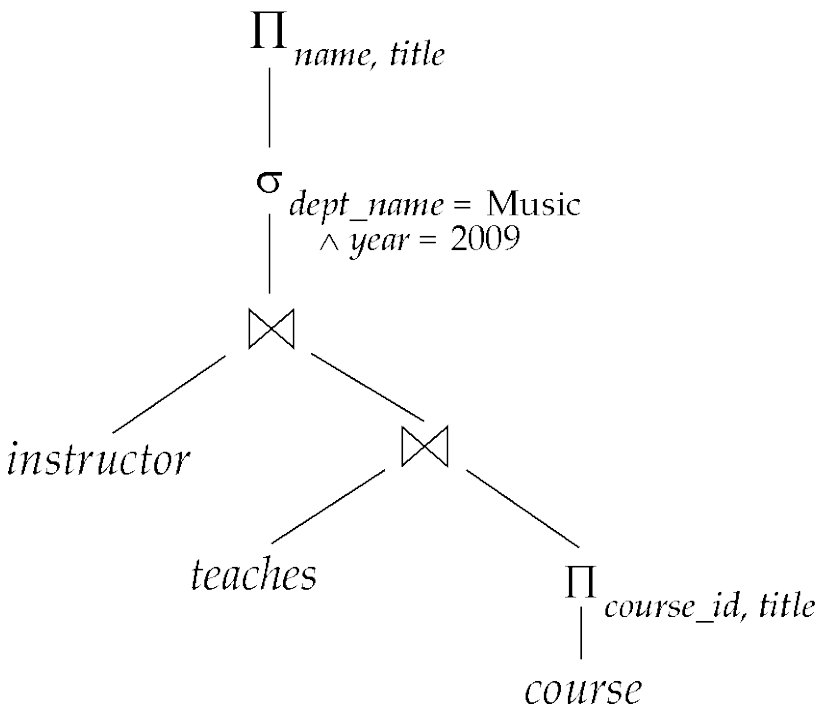
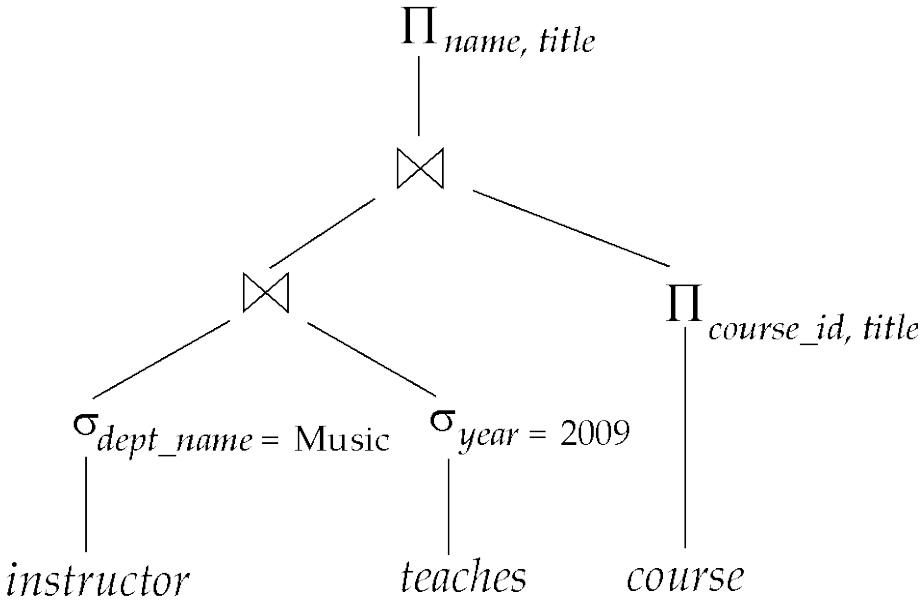

Rule 5
θ
E1
E2
θ
E2
E1
Rule 6.a
E3
E1
E2
E1
E2
E3
Rule 7.a
σθ
E1
E2
If θ only has attributes from E1
σθ
E2
E1



Figure 13.04



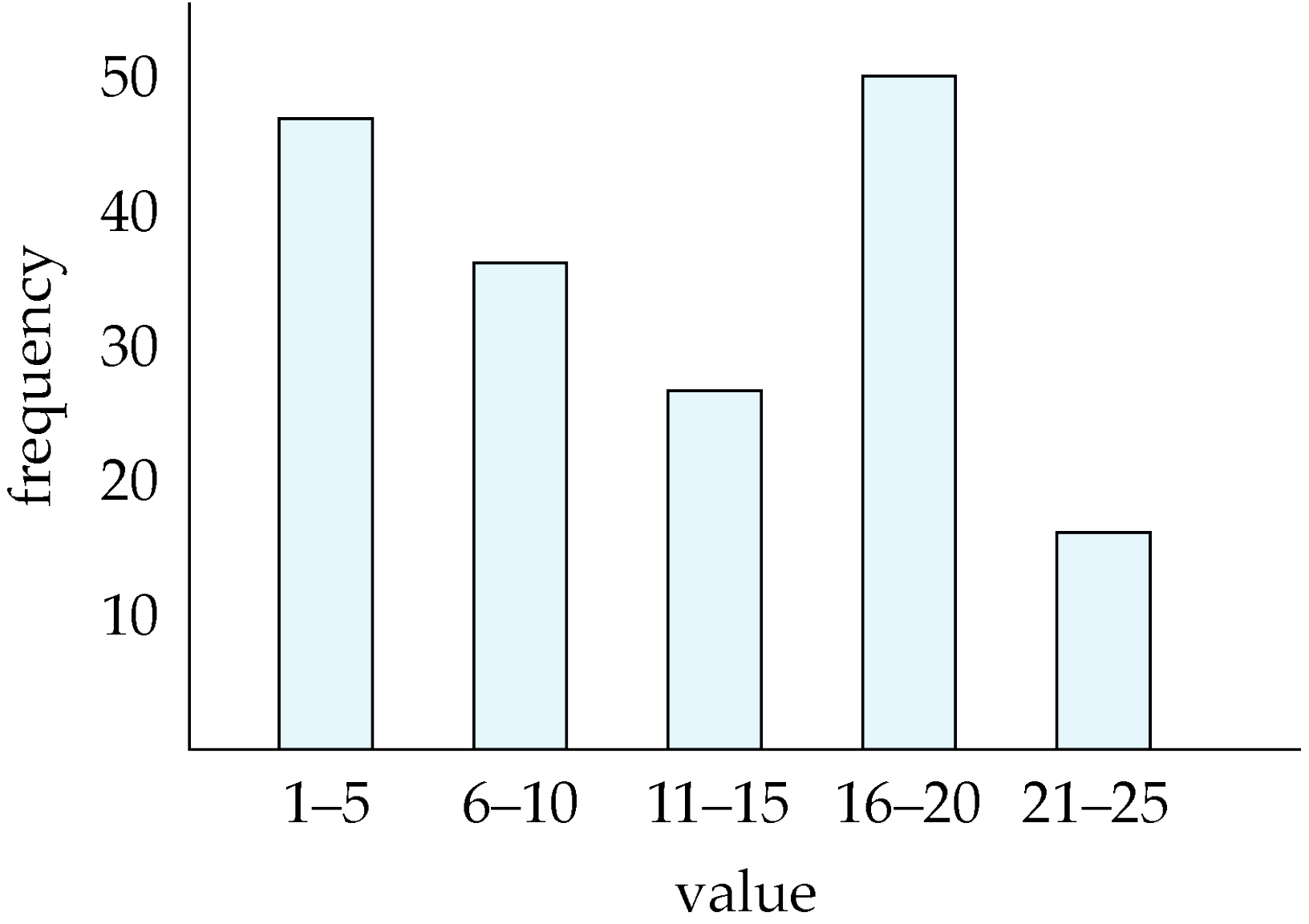
(a) Initial expression tree



(b) Tree after multiple transformations



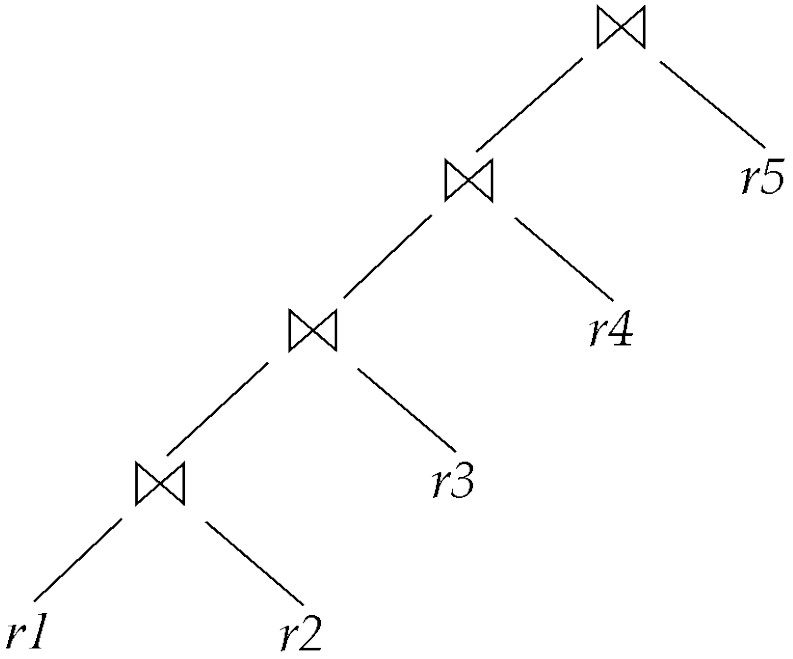
Figure 13.06



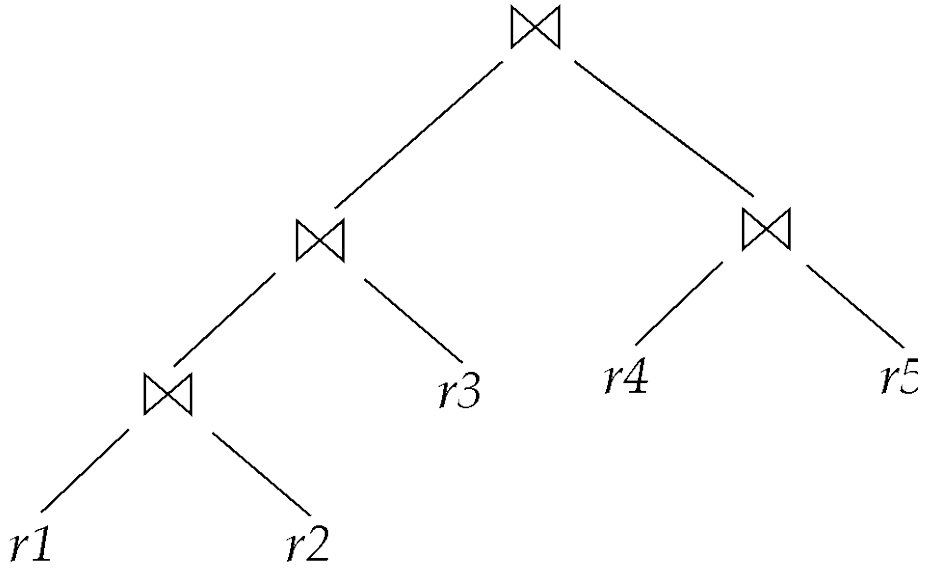
 ©Silberschatz, Korth and Sudarshan



Figure 13.08



(a) Left-deep join tree



(b) Non-left-deep join tree

©Silberschatz, Korth and Sudarshan